

Life Expectancy

Data Science Project

Made By:

Chris Akram 23-101219

Ziad Mohamed 23-101280

Abstract

This technical report will cover a lot on the person's Life Expectancy based on a few factors that will be covered. The objective of this project was to try and build a model able to detect and predict the expected life expectancy of people from different countries based on some features. The main problem that was faced is that not all countries have the same life expectancy as each other due to multiple factors that are unique to that country so it was attempted to try and build a model that can predict the life expectancy of any country accurately enough using the medical factors and **income composition of resources(ICOR)** as the main factors in our project building.

The model that was built showed promising results and findings in a few features that were used, it was found out that three features were the most important using Linear Regression and Polynomial Regression which were **Adult Mortality, ICOR, Schooling**. ICOR and Schooling showed **excellent** accuracy and **good or acceptable** accuracy respectively for our testing data using the R^2 for linear regression as the data clearly showed that as the ICOR or the Schooling of a country increased it **DIRECTLY** correlated with how long a person is to live even if the schooling was only showed a low accuracy result the graphs in the **results** section showed that it was an acceptable or even a good example of linear regression. Adult mortality on the other hand wasn't that perfect despite its near similar test accuracy with about **good or acceptable** accuracy using polynomial regression of **High Power** to achieve regression somewhat logical, following what was said in linear regression it is only logical to say that adult mortality also nearly directly affects the life expectancy however, as will be shown in the graph in the **results** section the graph of adult mortality got a lot of outliers which may make adult mortality not as correlating to life expectancy as our other two features.

The model also used classification to show which countries whether **developing** or **developed** were used to see how life expectancy would be categorized in developing countries and developed countries and if the country is developed does that mean its life expectancy is higher or not. The results showed that developed countries tend to have higher life expectancy rates than developing countries due to their higher ICOR and Schooling than the developing countries thus directly affecting their mortality rates.

After explaining the main things that were visualized and taken into account, it can be said that life expectancy is directly related to ICOR which is part of the human development index, this means that the higher the welfare of the person the longer they are expected to live and also schooling was a huge factor as countries with good education showed a trend of higher life expectancy. Finally, countries who want to increase their country's life expectancy should invest more in education, schooling and ICOR as they are the main factors affecting this matter.

Introduction

- **Background and Context:**

Life expectancy is a statistical measure that indicates the average number of years an individual is expected to live based on current mortality rates. This measurement is often utilized to assess and compare the health and economic conditions of populations, as it can highlight disparities between different geographic areas, ethnic groups, and socioeconomic statuses.

Factors influencing life expectancy include healthcare quality, environmental conditions, and lifestyle behaviors such as smoking or physical activity.

Life expectancy is important because it serves as a key indicator of a population's overall health and well-being.

Literature Review: Factors Influencing Life Expectancy: Global Evidence from 125 Countries

Link:

https://www.researchgate.net/publication/391397521_Factors_Influencing_Life_Expectancy_Global_Evidence_from_125_Countries

This study examines the relationship between life expectancy and eight key variables that strongly affect it using applications like personal correlation and multiple linear regression analysis. The study uses a similar data set to the one used in this project.

It uses a WHO data set that includes socioeconomic, demographic, healthcare, and lifestyle variables.

This study yielded findings about the predictors that influence life expectancy such as alcohol consumption, happiness, carbon dioxide emissions, and air pollution.

Some other factors such as BMI, GDP, the corruption perception index, and government health expenditure did not emerge as statistically significant predictors in the multiple regression analysis used in this study.

The aforementioned factors' influence cannot be entirely discounted as they still play a role in shaping life expectancy which will be clear if they're examined through different methodological approaches such as non-linear relationships that may provide deeper insights of their roles in determining life expectancy.

Project Objectives:

This project aims to train a regression model on expecting life expectancy trends based on global data of different diseases rates, schooling rates, and mortality rates using two different regression models : linear regression, and polynomial regression.

The project also aims to train a classification model based on the same data and the same data features to classify the values of each feature whether it's of a developing country or a developed country using two different classification models : KNN, and logistic classifier.

Materials And Methods

- Data Description:

In this project one single data set was used, this data set was taken from kaggle based on [Life Expectancy \(WHO\)](#) from [Kaggle](#). The data set has different countries from around the world in different years meaning that one country is repeated multiple times in different year spans with different values in each different features the dataset offers. Our dataset also had many variables and features which had different data types and are in total 22 and they are:

1. Country	object
2. Year	int64
3. Status	object
4. Life expectancy	float64
5. Adult Mortality	float64
6. infant deaths	int64
7. Alcohol	float64
8. percentage expenditure	float64
9. Hepatitis B	float64
10. Measles	int64
11. BMI	float64
12. under-five deaths	int64
13. Polio	float64
14. Total expenditure	float64
15. Diphtheria	float64
16. HIV/AIDS	float64
17. GDP	float64
18. Population	float64
19. thinness 1-19 years	float64
20. thinness 5-9 years	float64
21. Income composition of resources	float64
22. Schooling	float64

The different features are spread along 3 major datatypes: float64,int64 and object, we got **16** float64, **4** int64 and finally **2** objects. This data set was also structured as a CSV file downloaded from Kaggle.

Data Cleaning

While working on this data it found out that there were 22 different features spread along 22 different columns, each column with its own data type and thousands of rows, it was decided to focus our model to only care about the health aspects affecting the life expectancy as well as the human development index and not the economic aspects given in the data we are using, all other features were dropped. The features dropped include:

1. Total Expenditure
2. GDP
3. Percentage Expenditure
4. Population

These features were dropped as they were focused on the economic side relative to life expectancy and population was also dropped as it has no correlation with the life expectancy of a person.

After the features that didn't suffice were removed from our model the decision was made to move towards dealing with nulls and empty spaces or illogical zeros, This process started by taking care of the nulls first. The method that was taken to take care of the nulls was that if any row got less than **50** nulls that row would be completely removed from our dataset, this method was performed on exactly **7** features of the remaining **18**. As for why this method was taken is because our data consists of about **2938** unique rows thus removing a fraction of that which is less than 50 wouldn't really affect our model. The features that had rows dropped were:

1. Life Expectancy
2. Adult Mortality
3. BMI
4. Polio
5. Diphtheria
6. Thinness 1-19 Years
7. Thinness 5-9 years

Now we had **4** features that got empty cells, to deal with those empty cells and since the data got the same country repeated multiple times along different year spans it was decided that building a function called smart impute is the best approach. How this function works is that if there is a blank or empty cell in a cell the model fills it up by first checking if the country got relevant values from previous or future years and get the mean of that, if that data isn't available then the year in general is checked and its mean is taken and finally if that isn't available then the country status whether it's developed or not is taken into consideration and finally if none of that is available the column mean is taken instead. By doing this the data is supposedly clean, however a problem was encountered in the features **income composition of resources** aka a

core aspect of **human development index** and the feature **schooling** and to fix that we will have to add some fixes on top of our smart impute.

The features income composition of resources which we will name (**ICOR**) from now on for short and schooling were found to have illogical zeros in their columns, such a thing would make the data inaccurate or illogical as it's near impossible to have ICOR to be equal to zero and same goes for schooling. After learning that information all zeros in those two rows were then turned into empty spaces then passed into our smart impute function which was talked about in the previous paragraph to get the mean and be put in those empty spaces.

- Methodology

After explaining how the data cleaning techniques were done, now it's time to move forward to explain all the methodologies and techniques used to display our data as well as the regression and classification models that were chosen and why they were chosen.

Evaluation Metrics

The first thing that will be discussed is the model evaluation metrics, only two metrics were used and they were only used in the regression models. These metrics are the **MSE** and **R²**. First the **MSE** also known as **Mean Squared Error** is used to identify whether the model used gives a far fetched prediction or over/under fitting the data. The bigger the **MSE** gets the worse the model performance is. The formula for MSE is:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

N means number of data points

Y means observed value

Y(bar) means predicted value

Second metric used was the **R²** also known as **coefficient of determination** it gives a fraction of the variance in the data explained by the model and also indicates how good a model is and usually is between 0 and 1 unlike the **MSE** and its formula is:

$$r^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} = 1 - \frac{MSE}{\sigma_Y^2}$$

With the same variables as the **MSE**. Now that the metrics used were explained it's time to dive deeper into regression and classification and eventually the visualization used.

Regression Models and Explanation

The second thing that will be discussed is the types of regression and classification methods used in detail, first on the list is **Regression**:

Before we explain the regression models that were chosen, it's a must to explain what regression is. In quick and simple terms regression in data science is a machine learning technique that is used to predict continuous values against features to try and predict values of similar properties on that model and evaluate them, now we move on to explain the regression models that were used.

The regression model that was decided to be used to attain up most efficiency was linear regression, it was decided on since **two** of our features are nearly perfect examples of the linear regression model and showed promising results in the **MSE** and **R²** scores in both training and testing phases, the features were **Schooling** and **ICOR** against **Life Expectancy**.

The second regression model we used was polynomial regression as **one** feature was not good enough to be linear regression but showed promising results as the degree of the polynomial increased till it was decided that the degree of polynomial regression will be **six**, it gave an **acceptable** result for both **MSE** and **R²** scores for both the training and testing phases the feature was **Adult Mortality** against **Life Expectancy**.

The reason why those two types of regression were selected is because when evaluating the data to figure out which technique to use, it was seen that the **two** features that were used showed a linear relationship against the **Life Expectancy** in which in linear regression **Schooling** and **ICOR** were clearly linear from the graph and the feature used in polynomial regression which was **Adult Mortality** wasn't a near perfect linear relationship with **Life Expectancy** but it gave a good enough result for the **sixth** degree polynomial regression, the other **fifteen** features showed no real correlation to any regression type and were random and all over the graph thus the decision was made to use the mentioned features against our two regression models linear and polynomial.

Classification Model and Explanation

Now that we have discussed the why of our regression models and explained them and why the some features were included into them, it's now time to move forward into the third thing on our list **Classification**:

Before diving into classification it is a must to explain what classification is like how it was done with regression. Classification in data science is a machine learning technique that assigns labels on new data points based on previous categorical data used before that were labelled from training data and now we move on to explain the classification methods that were used.

The first classification that was chosen was **K-Nearest Neighbors(KNN)**, to use this classification method **three** features were chosen to be categorized against the **Status** of a country whether **Developed** or **Developing**. The features that were chosen are **Life Expectancy, ICOR, Schooling**. Again **ICOR** and **Schooling** are chosen. A 3D map was done between all features and **Status** and correlation between them were very apparent as it showed clearly that our features had a huge impact on whether the country was a developed or developing country.

The second classification method that was chosen was **Logistic Regression Classifier**, the same features as the **KNN** were used to build a confusion matrix to see whether the model correctly predicted the **status** of our countries and check for accuracy and high accuracy was seen as our model had about **90%** accuracy showing that it classified the test data correctly.

The reason why those two types of classification were chosen and those features were because after testing them those features showed the most correlation to the **Status** were the ones that were chosen and using the **KNN** and **Logistic Regression Classifier** it was further proved that the **Status** of the country can be classified from those three features.

Data Visualization Techniques

After explaining the metrics, regression and classification that were used it's time to state how we visualized the data and what methods and techniques were used.

Firstly, the imported **CSV** table of **Life Expectancy**. It was obtained from **Kaggle** and was displayed using the pandas library from python as they are like tabular data that store all the info we need and we can access it freely using the correct syntax as it organizes data in columns and rows. Pandas were used since it was a table format and contained many columns and rows that needed to be seen at first to determine what features that were going to be dropped and to do the eye test and check if some features had multiple missing values to later on decide how to clean them.

Secondly, the **Linear Regression** and **Polynomial Regression** models were made using a **Scatter plot** which contain 2 axis but instead of a line or bar its points on the graph displaying data points and is used to display our data as well as the predicted data which was displayed linearly on top of the scatter plot to show whether the model followed linear regression or not. Scatter plots were chosen since the data had multiple different data points that needed to be addressed thus the scatter plot was the perfect solution.

Thirdly, the **KNN** also used **Scatter plots** however, it was a 3D scatter plot as **three** features were being addressed against the **Status** of the country and the scatter plot made perfect sense since it displayed all the data points in a 3D plot with three different axes showing exactly how the features were correlated to **Status** of the country.

Finally, **Logistic Regression Classifier** this classification technique used a confusion matrix which is also a 2D matrix with 2 axes, one axis defines the true actual data in our case is the Y-axis the other displays the predicted data which is the X-axis split into 4 spaces each space is addressed. As for why it was used it is because the confusion matrix showed the true positive, negative and false positive, negatives put into our 4 spaces of confusion matrix to show how accurate it was as well as it gave accuracy and precision of our models using a few formulas which will be stated below thus showing exactly how many data points were classified correctly and how many were not.

Formulas:

Precision

$$\frac{TP}{(TP + FP)}$$

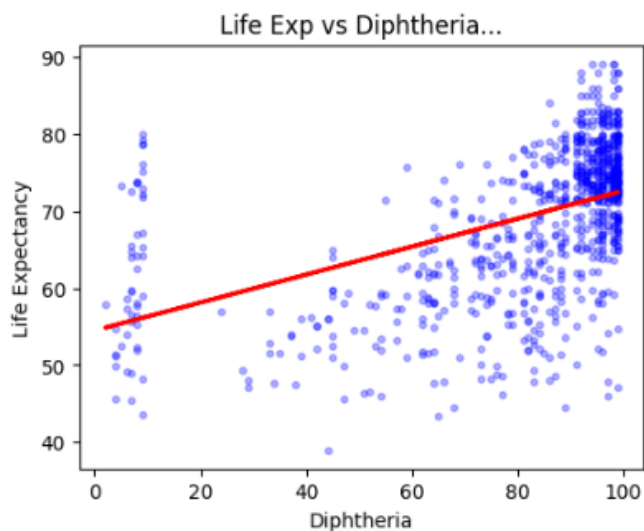
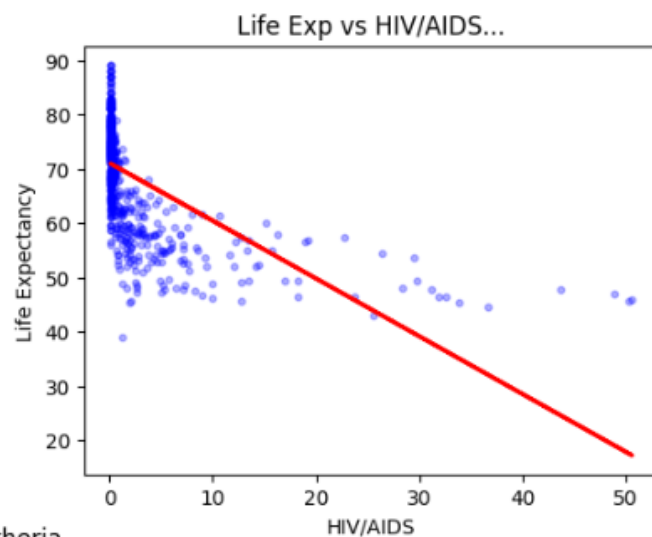
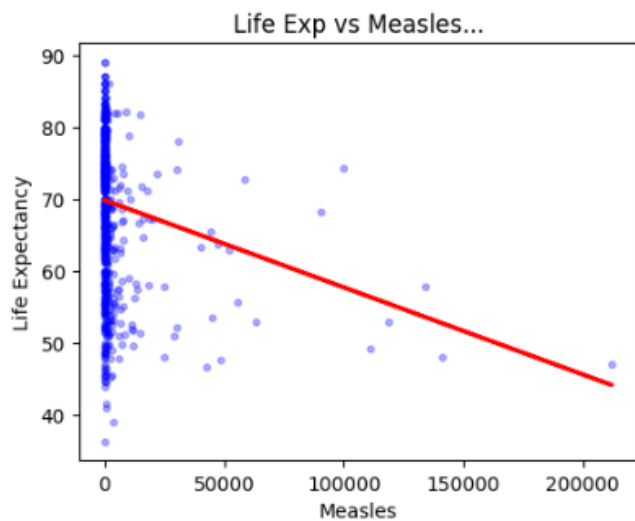
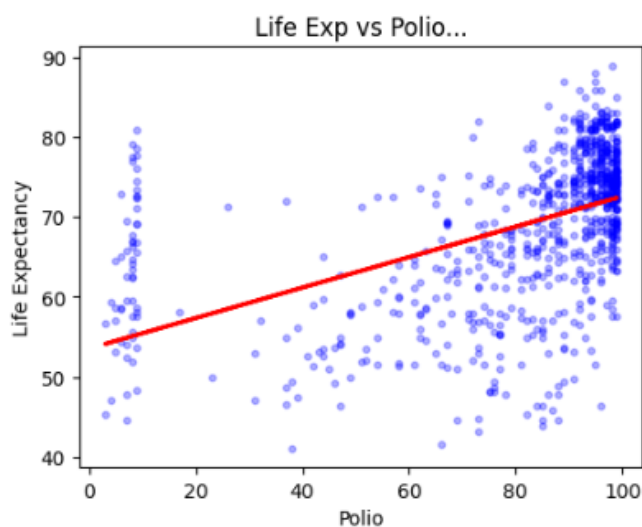
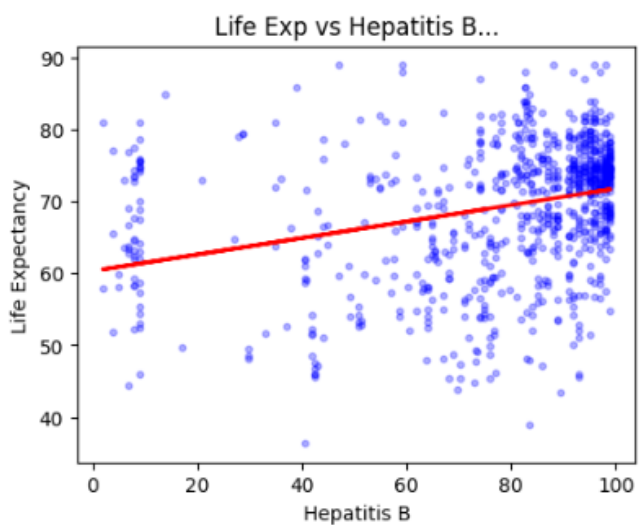
Accuracy

$$\frac{TP + TN}{(TP + TN + FP + FN)}$$

Results

- Key Findings: Regression

linear regression: it's noticed that all of the disease features do not correlate well linearly with life expectancy as shown in the following graphs:



Disease features did not perform well in the MSE nor the R-Squared scores:

```
Testing Feature: Hepatitis B
R-squared Train: 0.0848
R-squared Test Score: 0.0938
Mean Squared Error (MSE) Train: 83.9348
Mean Squared Error (MSE) Test: 78.3255
-----
```

```
Testing Feature: Measles
R-squared Train: 0.0189
R-squared Test Score: 0.0356
Mean Squared Error (MSE) Train: 85.7261
Mean Squared Error (MSE) Test: 92.8304
-----
```

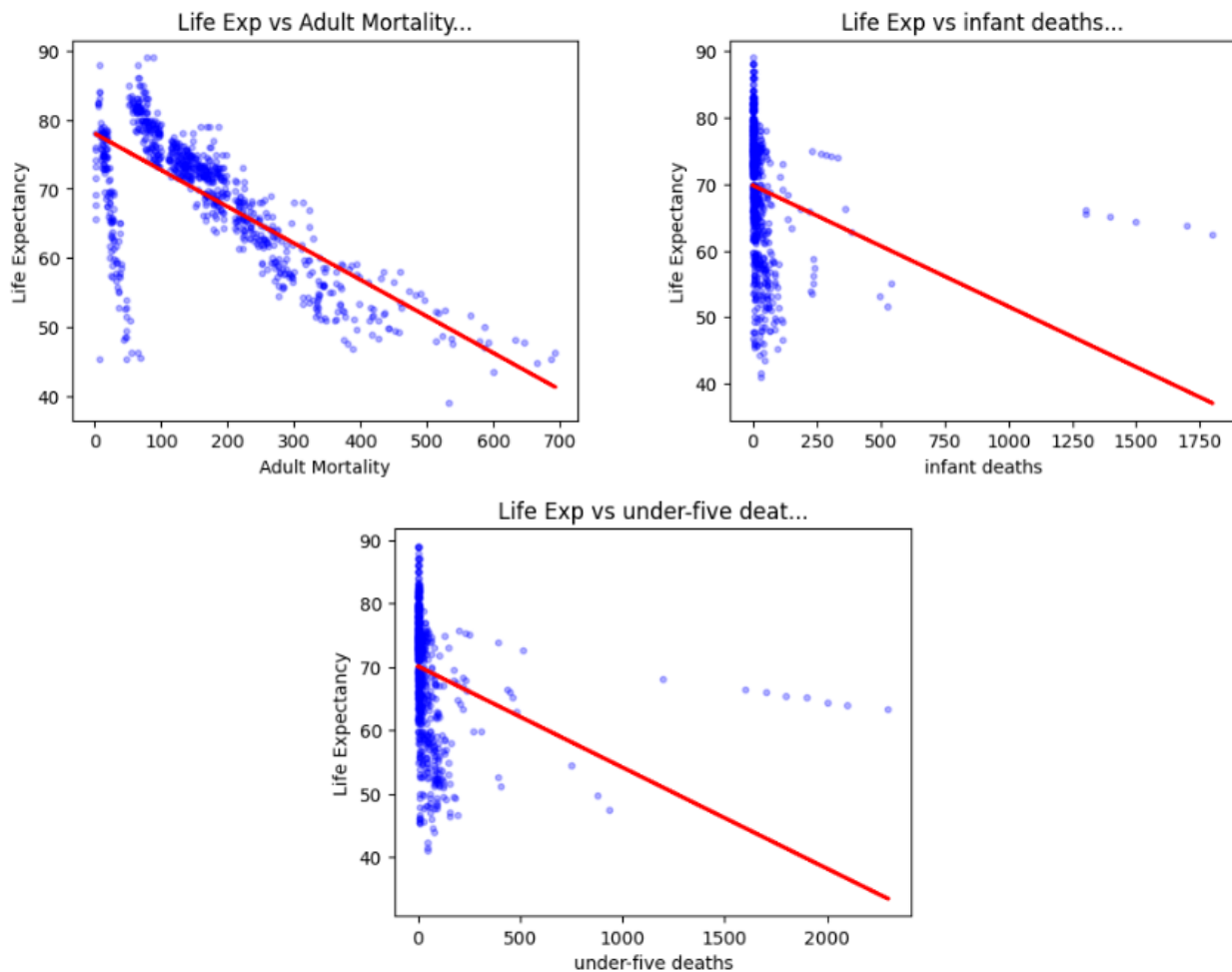
```
Testing Feature: Polio
R-squared Train: 0.2098
R-squared Test Score: 0.2290
Mean Squared Error (MSE) Train: 71.8145
Mean Squared Error (MSE) Test: 68.1557
-----
```

```
Testing Feature: Diphtheria
R-squared Train: 0.2091
R-squared Test Score: 0.2689
Mean Squared Error (MSE) Train: 70.2720
Mean Squared Error (MSE) Test: 68.0155
-----
```

```
Testing Feature: HIV/AIDS
R-squared Train: 0.3174
R-squared Test Score: 0.3047
Mean Squared Error (MSE) Train: 61.4997
Mean Squared Error (MSE) Test: 62.6776
-----
```

The high MSE scores and low R-Squared scores in the training phase of those disease features are due to inconsistency of their data which is due to temporal and geographic confoundings, as disease trends depend heavily on time, region and development status.

Mortality rates also do not correlate well linearly with life expectancy (except for Adult Mortality) as shown in the following graphs:



The scale difference between the infant and under five mortality graphs compared to the adult mortality graph is due to the fact that the child mortality rates have extreme outliers that stretches the scale significantly.

Those countries' child mortality rates are very high due to several factors like extreme poverty, weak healthcare systems, malnutrition, and poor sanitation and water access.

Mortality rates MSE and R-Squared scores:

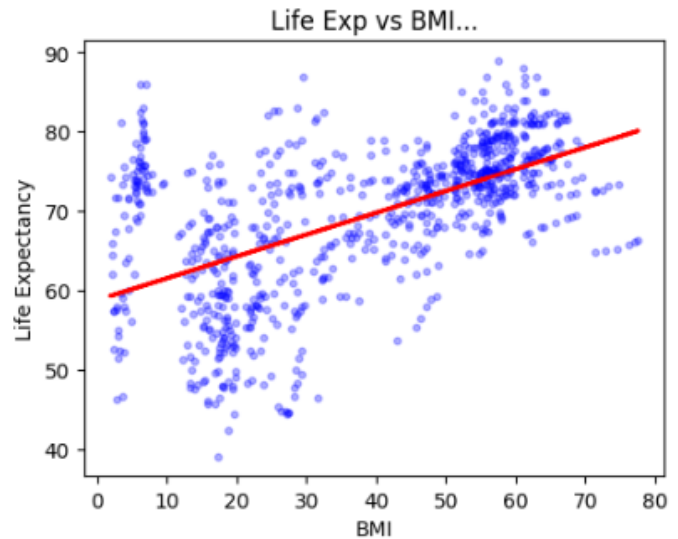
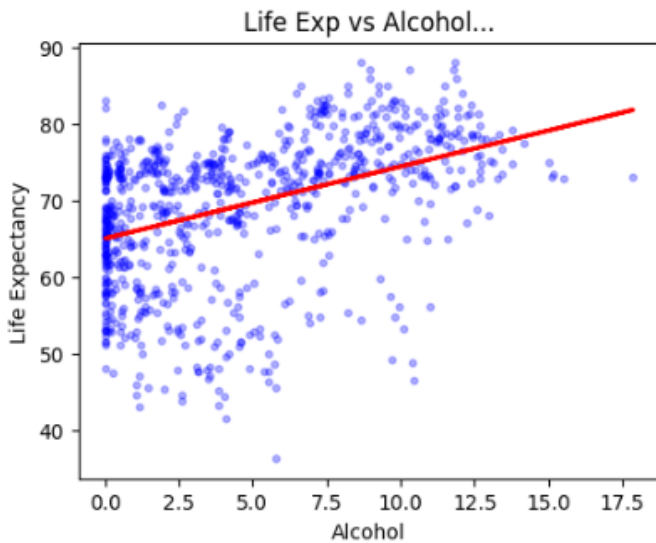
```
Testing Feature: Adult Mortality
R-squared Train: 0.4681
R-squared Test Score: 0.5105
Mean Squared Error (MSE) Train: 48.5964
Mean Squared Error (MSE) Test: 42.6910
-----
Testing Feature: infant deaths
R-squared Train: 0.0461
R-squared Test Score: 0.0179
Mean Squared Error (MSE) Train: 85.7835
Mean Squared Error (MSE) Test: 88.8819
-----
Testing Feature: under-five deaths
R-squared Train: 0.0614
R-squared Test Score: 0.0133
Mean Squared Error (MSE) Train: 85.1096
Mean Squared Error (MSE) Test: 87.6116
-----
```

Adult mortality's MSE and R-Squared scores show a moderate correlation between adult mortality and life expectancy, which is due to its data having a more concentrated linear distribution compared to the other two rates.

The two child mortality rates have high MSE scores and very low R-Squared scores due to the aforementioned reasons.

Habits rates. In this dataset we have only 2 habits columns: BMI (body mass index), and alcohol consumption

Strangely enough these two habit features seem to correlate positively with life expectancy as shown in the following graphs:



Why do alcohol consumption and BMI correlate positively with life expectancy?

As for alcohol consumption:

It's due to the fact that alcohol is mostly consumed in developed countries, which have good healthcare systems and cleaner atmosphere, more income and schooling rates and lower disease rates, and all these factors lead to a higher life expectancy than the developing countries.

As for BMI:

High and average BMI values usually indicate high life expectancy values, but very high BMI values can be lethal and that's when BMI starts to correlate negatively with life expectancy, so the relationship between them is likely to be U shaped.

Habits rates MSE and R-Squared scores:

```
-----  
Testing Feature: Alcohol  
R-squared Train: 0.1581  
R-squared Test Score: 0.1833  
Mean Squared Error (MSE) Train: 76.4361  
Mean Squared Error (MSE) Test: 72.3044  
-----
```

```
-----  
Testing Feature: BMI  
R-squared Train: 0.3318  
R-squared Test Score: 0.2963  
Mean Squared Error (MSE) Train: 60.5526  
Mean Squared Error (MSE) Test: 62.5959  
-----
```

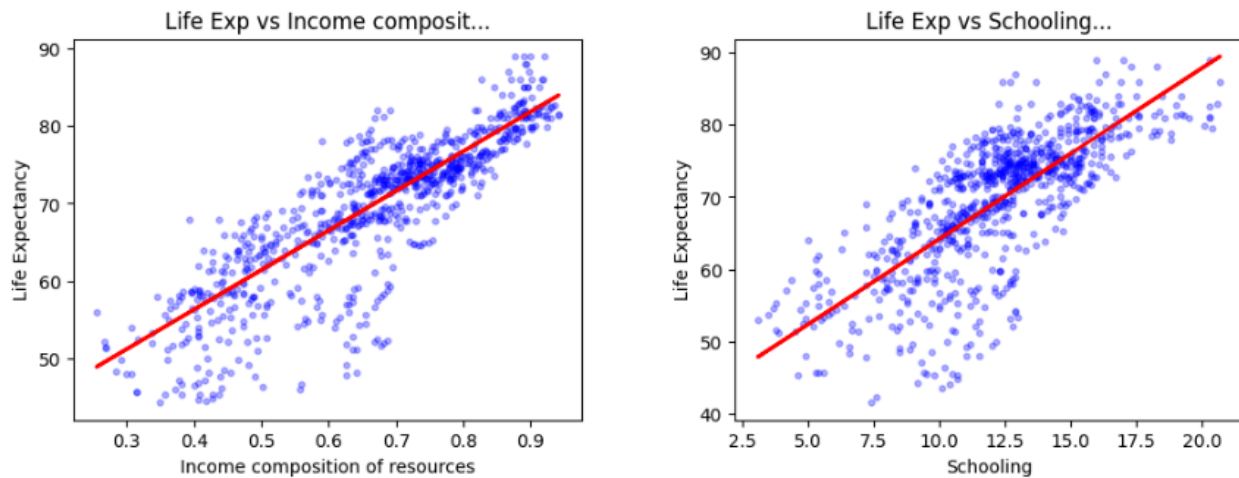
Although the graphs show a moderate linear fit, the MSE scores are high and R-Squared scores are low.

This is due to the high variance in both graphs.

For Alcohol: at an alcohol consumption level of roughly 2.5, life expectancy ranges anywhere from 45 to 80 years, which is a massive 35-year difference for the same amount of alcohol.

For BMI: at a BMI of roughly 20, life expectancy ranges from 45 to 85 years.

Schooling and ICOR rates graphs:



Schooling and ICOR rates MSE and R-Squared scores:

```
-----
Testing Feature: Income composition of resources
R-squared Train: 0.7219
R-squared Test Score: 0.7161
Mean Squared Error (MSE) Train: 25.1235
Mean Squared Error (MSE) Test: 25.4447
-----

Testing Feature: Schooling
R-squared Train: 0.5626
R-squared Test Score: 0.5582
Mean Squared Error (MSE) Train: 40.4853
Mean Squared Error (MSE) Test: 37.2861
-----
```

ICOR showed the strongest linear relationship with life expectancy among all 14 features tested. The high R-Squared scores show that countries with more equitable resource distribution consistently have higher life expectancy.

The fact that training and test R-Squared scores are nearly identical means that the model generalizes the relationship between ICOR and life expectancy well.

Schooling also showed a strong linear correlation with life expectancy, though slightly weaker than ICOR. The reason for strong correlation is that more education rates lead to better health literacy and higher income potentials which lead to improved access to healthcare. Like ICOR, the close match between train and test scores of R-Squared suggests that this relationship between schooling and life expectancy is robust and reliable, and the model is able to generalize the schooling data.

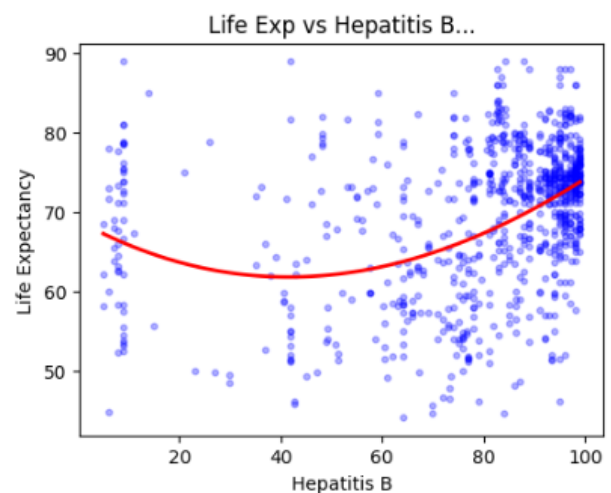
Polynomial Regression: Linear regression was not sufficient to describe a lot of features like the disease rates for example, due to the fact that these features exhibit non-linear patterns that cannot be captured by a simple linear regression.

To fix this problem, a polynomial regression was employed to the same data with different degrees depending on which degree suits which feature better.

Polynomial degrees ranging from 2 to 10 were all tested on all features and each feature's optimal degree was selected based on the highest R-Squared scores and the lowest MSE scores, which allowed the model to be more accurate with its predictions.

Disease features with polynomial regression: 1) Hepatitis B

```
-----  
Testing Feature: Hepatitis B  
R-squared Train: 0.1509  
R-squared Test Score: 0.1608  
Mean Squared Error (MSE) Train: 78.6215  
Mean Squared Error (MSE) Test: 70.6384  
-----
```



The polynomial regression model for Hepatitis B vaccination rates reached its optimal performance at degree 3, beyond which no significant improvement was observed. The model demonstrated weak predictive capability with R-Squared values of 0.1509 for training and 0.1608 in testing, indicating that Hepatitis B vaccination rates explain only approximately 15-16% of the variance in life expectancy.

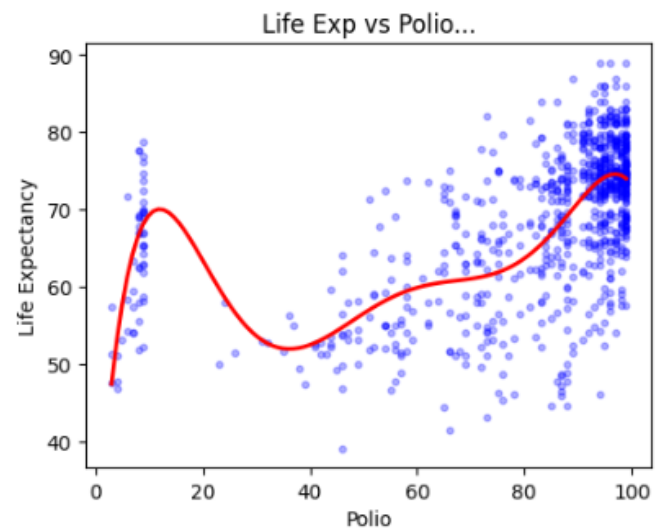
The MSE values were relatively high at 78.62 for training and 70.64 for testing, suggesting substantial predictive errors. Notably the test set performed slightly better than the training set in both metrics, which may indicate that the model generalizes reasonably well despite its overall weak performance.

The saturation at degree 3 with these bad values of both MSE and R-Squared mean that Hepatitis B alone is not a strong predictor. This weak correlation aligns with the understanding that vaccination rates are often a consequence of a country's development status and healthcare systems rather than a direct determinant of life expectancy.

2) The Measles ideas feature's MSE and R-Squared scores only became worse when a polynomial regression model is applied to the feature, so it's not going to be discussed, as a linear regression model is the best to describe its relationship with life expectancy. The Measles feature is not a strong predictor of life expectancy.

3) Polio

```
-----
Testing Feature: Polio
R-squared Train: 0.4090
R-squared Test Score: 0.4099
Mean Squared Error (MSE) Train: 53.1519
Mean Squared Error (MSE) Test: 53.3885
-----
```



The polynomial regression model for Polio vaccination rates reached its optimal performance at degree 6, beyond which no significant improvement was observed.

The model demonstrated moderate predictive capability with R-Squared values of 0.4090 for training and 0.4099 for testing, indicating that Polio vaccination rates explain approximately 41% of the variance in life expectancy.

MSE values were moderate at 53.15 for training and 53.39 for testing, suggesting reasonable predictive accuracy. Notably, the training and test sets performed very similarly in both metrics, which indicates that the model generalizes well without overfitting.

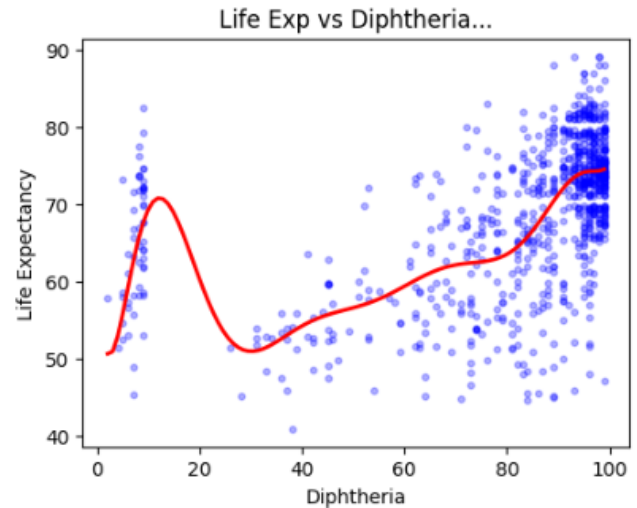
The saturation at degree 6 with these moderate values of both MSE and R-Squared means that Polio vaccination rates are a reasonably strong predictor, though not as strong as other features like Schooling or ICOR.

This moderate correlation aligns with the understanding that polio vaccination rates are closely tied to a country's healthcare infrastructure and development status, which are key determinants of life expectancy.

The need for a higher-degree polynomial (degree 6) compared to other features suggests that the relationship between polio vaccination and life expectancy is more complex and non-linear.

4) Diphtheria

```
-----  
Testing Feature: Diphtheria  
R-squared Train: 0.4137  
R-squared Test Score: 0.4131  
Mean Squared Error (MSE) Train: 53.1447  
Mean Squared Error (MSE) Test: 52.1596  
-----
```



The polynomial regression model for Diphtheria vaccination rates reached its optimal performance at degree 10, beyond which no significant improvement was observed.

The model demonstrated moderate predictive capability with R-Squared values of 0.4137 for training and 0.4131 in testing, indicating that Diphtheria vaccination rates explain only approximately 41% of the variance in life expectancy.

MSE values were 53.1447 for training and 52.1596 for testing, suggesting moderate predictive errors.

Notably, the test set performed well, which may indicate that the model generalizes reasonably well despite its overall moderate performance.

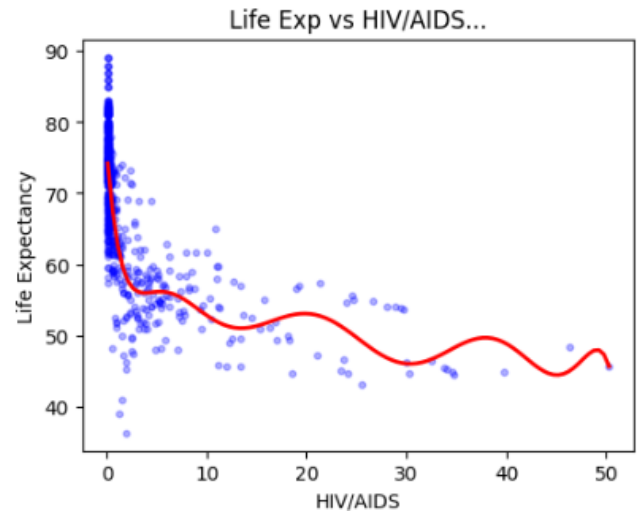
The saturation at degree 10 with these values of both MSE and R-Squared mean that Diphtheria vaccination rates alone is not a strong standalone predictor.

This aligns with the broader understanding that vaccination coverage is typically a downstream consequence of a country's healthcare infrastructure, public health policy, and overall socioeconomic development rather than a direct, isolated determinant of life expectancy.

The plateauing at a higher polynomial degree further suggests that the relationship is inherently noisy or heavily confounded by other systemic factors, meaning that forcing a higher-degree polynomial does not capture a stronger underlying causal pattern.

5) HIV/AIDS

```
-----  
Testing Feature: HIV/AIDS  
R-squared Train: 0.6445  
R-squared Test Score: 0.6877  
Mean Squared Error (MSE) Train: 31.2983  
Mean Squared Error (MSE) Test: 29.6595  
-----
```



HIV/AIDS stands out as the feature where polynomial regression provided the most significant performance boost compared to a simple linear approach.

While the Linear Regression model struggled to capture the data's complexity, managing a modest R^2 of only 0.31 and a high MSE of 61.40 introducing a high-degree polynomial completely transformed the results.

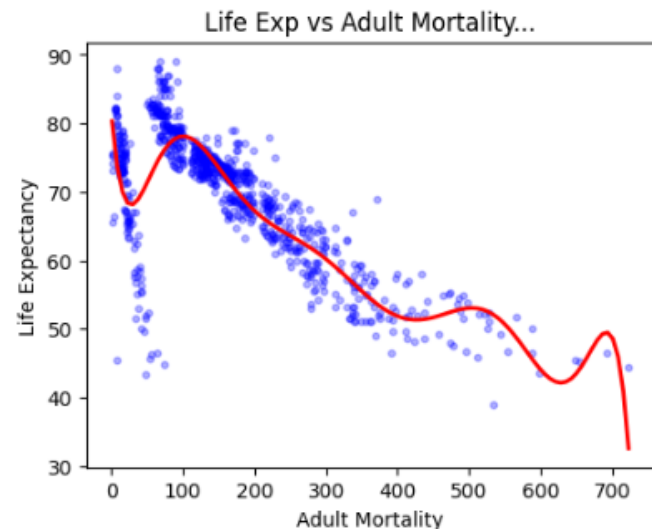
By fitting a 9th-degree polynomial, the model's explanatory power effectively doubled, achieving a Test R^2 of 0.6877 (explaining nearly 69% of the variance) and slashing the MSE down to 29.66. This dramatic improvement confirms that the relationship between HIV/AIDS prevalence and life expectancy is highly non-linear.

The model captures a complex curve where life expectancy drops precipitously at the onset of the epidemic, followed by nuanced fluctuations at higher prevalence rates.

Notably, the Test set actually outperformed the Training set (0.6877 vs 0.6445), suggesting that despite the high polynomial degree, the model generalizes exceptionally well without overfitting.

Mortality features with polynomial regression: Adult Mortality

```
Testing Feature: Adult Mortality
R-squared Train: 0.6372
R-squared Test Score: 0.6903
Mean Squared Error (MSE) Train: 32.9602
Mean Squared Error (MSE) Test: 27.3146
-----
```



Polynomial regression demonstrates significantly superior performance in modeling the relationship between adult mortality and life expectancy compared to linear regression.

While the linear model achieved modest R-squared scores of 0.47 for training and 0.51 for testing with MSE values of 48.60 and 42.69 respectively, the polynomial regression model at degree 9 substantially improved these metrics, achieving R-squared scores of 0.64 for training and 0.69 for testing, with MSE values dropping to 32.96 and 27.31 respectively.

The degree 9 polynomial effectively captures the non-linear relationship between adult mortality and life expectancy by modeling several distinct patterns in the data: a sharp initial decline in life expectancy as mortality rates increase from 0 to 200 per 1,000 population, followed by a plateau region, and subsequent fluctuations in the 400-700 mortality range.

This higher-degree polynomial successfully fits the local variations and curvature in the data that the rigid linear approach fails to capture, particularly the dense clustering of data points at lower mortality rates and the scattered outliers at higher mortality values.

The improvement in both R-squared and MSE metrics confirms that adult mortality impacts life expectancy through a complex, non-linear pattern rather than a constant linear trend.

The polynomial model's ability to adapt to the data's curvature while maintaining reasonable generalization (as evidenced by the improved test performance) makes it the superior choice for predicting life expectancy based on adult mortality rates.

The child mortality rates (infant deaths and under-five deaths) were excluded from polynomial regression modeling due to extreme outliers that significantly distort the data distribution, resulting in poor model fit as evidenced by high MSE and very low R-squared scores.

Habits features with polynomial regression:

BMI and alcohol consumption were excluded from the polynomial regression analysis despite their moderate performance in the linear regression model. This decision was made because both features exhibited a counterintuitive positive correlation with life expectancy in the linear model, which contradicts established medical and public health evidence.

Schooling and ICOR with polynomial regression:

For Schooling and ICOR, it's observed that model performance metrics (MSE and R-Squared scores) stabilized at degree one, with no statistically significant improvement observed for degrees two through ten. This suggests that the underlying relationship between these predictors and life expectancy is well-captured by a linear function. As such, we concluded that linear regression is the most appropriate model for these features, so they were excluded from polynomial regression analysis.

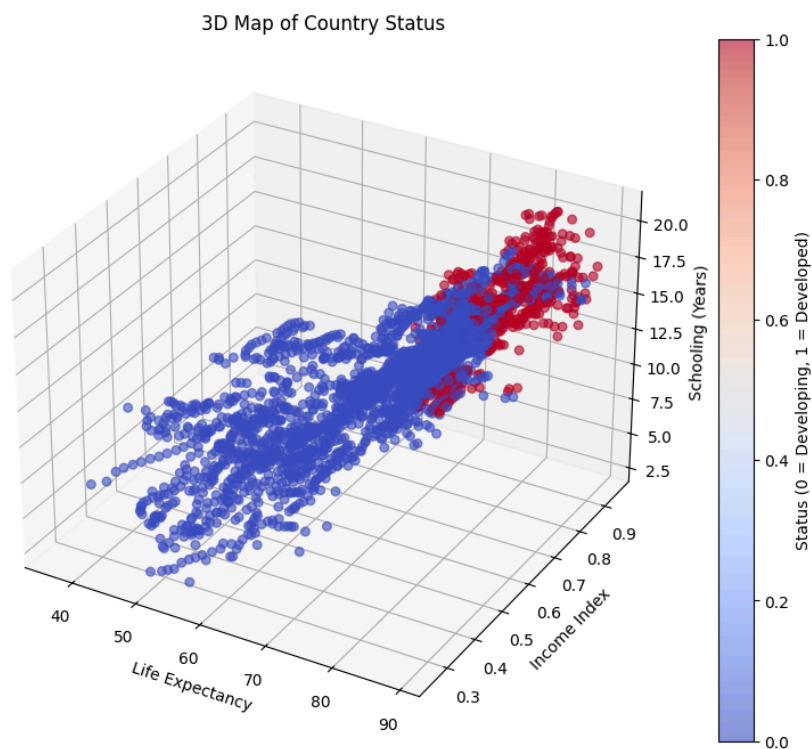
Classification:

Based on the linear regression analysis conducted in the previous section, ICOR and Schooling emerged as the two strongest linear predictors of life expectancy.

These variables directly capture a country's socioeconomic development and human capital investment, making them highly interpretable and logically aligned with the target classification task (Developed vs. Developing status).

By restricting the classification models to these two features, dimensionality and multicollinearity are reduced ensuring the model focuses on the most economically and educationally relevant drivers of development status.

KNN classifier: Accuracy: 91%



The 3D scatter plot visualizes the decision boundary in the feature space defined by life expectancy, ICOR, and Schooling, with color representing country status.

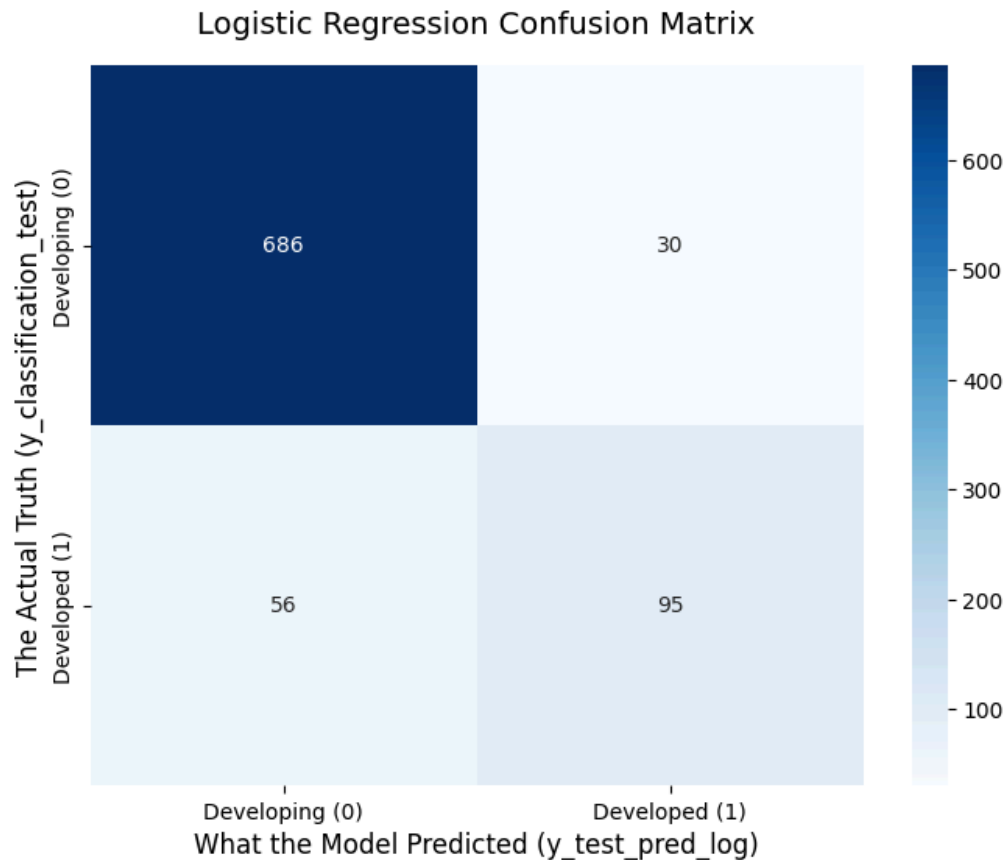
The visualization reveals a clear spatial separation: developed countries (red) cluster in the high-life-expectancy, high-income, and high-schooling region, while developing countries (blue) occupy the lower-to-mid range across all three axes.

The KNN classifier achieved an accuracy of 91%, closely mirroring the logistic regression results. The tight clustering and minimal overlap in the 3D space confirm that the selected features are highly discriminative. The few misclassifications likely represent countries in economic transition or those with unique demographic/healthcare profiles that deviate from the global trend.

LRP Classifier:

Train accuracy: 89.21%

Test accuracy: 90.08%



The model achieved a training accuracy of 89.21% and a test accuracy of 90.08%, indicating strong generalization with minimal overfitting.

Accuracy of 90.08% confirms the model reliably separates the two classes.

The dataset contains significantly more developing countries, which explains the higher True Negative rate (686) compared to True Positives (95).

The model shows a higher False Negative rate (56 developed countries misclassified as developing), likely due to transitional economies that exhibit high life expectancy but lag in income or schooling metrics.

Derived Metrics:

Precision (Developed): 76%

Recall (Developed): 62.9%

Precision (Developing): 92.4%

Recall (Developing): 95.8%

This indicates the model is highly confident in identifying developing nations but slightly conservative in classifying developed ones, which is a normal occurrence in imbalance classification tasks.

Conclusion

This project successfully built models to predict life expectancy and classify a country's development status using global health and economic data. Through our analysis, it became clear that Income Composition of Resources (ICOR) and Schooling are the strongest and most reliable linear predictors of how long people will live.

While disease rates did not show a simple straight-line relationship with life expectancy, applying polynomial regression proved that factors like Adult Mortality and HIV/AIDS have strong, complex impacts that drastically affect lifespan.

Additionally, our classification models, K-Nearest Neighbors and Logistic Regression, achieved around 90% accuracy in separating developed and developing nations based purely on their health, wealth, and education.

Based on these results, the clearest recommendation for any developing nation looking to improve its life expectancy is to heavily invest in its education systems and work to increase its overall ICOR. The data clearly shows that wealth distribution and schooling go hand-in-hand with better health literacy and access to healthcare, which directly extends human life.

However, this study did face a few limitations.

The dataset contained extreme outliers, particularly in child mortality rates, which stretched the scale and made it difficult to fit them into our models.

Disease data was also somewhat inconsistent because illness trends depend heavily on the specific region and time period.

Furthermore, the dataset was imbalanced, featuring significantly more developing countries than developed ones.

This imbalance caused our classification models to be a bit conservative, sometimes misclassifying developed nations or countries currently in economic transition because their income or schooling didn't quite match their high life expectancy.

For future work, this project could be expanded by addressing the imbalanced data to improve the classification of developed countries.

Future research could also take a closer look at those specific transitional economies to understand why their unique demographic and healthcare profiles deviate from global trends.

Finally, finding better ways to handle the massive outliers in child mortality could help create an even more complete picture of what drives global life expectancy.

Acknowledgements

- Team Contributions

Chris Akram Melad Sedra (23-101219):

- Data Cleaning code
- Regression code
- DashBoard
- Abstract typing
- Materials and Methods typing
- Appendix typing
- PowerPoint

Ziad Mohamed (23-101280):

- Data Cleaning Code
- Classification Code
- Introduction typing
- Results typing
- Conclusion typing
- PowerPoint
- Video

- External Support

Support came from: Dr. Fatma Elshahaby and TA Malak Khaled Hassanin

Works Cited

KumarRajarshi. "Life Expectancy (WHO)." *Www.kaggle.com*, 2018,

www.kaggle.com/datasets/kumarajarshi/life-expectancy-who/data.

Teh, Weng Shin, et al. "Factors Influencing Life Expectancy: Global Evidence from 125

Countries." *International Journal of Built Environment and Sustainability*, vol. 12, no. 2,

University of Technology Malaysia, May 2025, pp. 17–28,

<https://doi.org/10.11113/ijbes.v12.n2.1294>.

Appendix

- AI Usage

We used AI in this project when we got stuck during coding parts and the next are the screenshots of prompts and answers of AI for our questions.

Prompt:

```
df_lifeExp.columns = df_lifeExp.columns.str.strip()
df_lifeExp_clean = df_lifeExp.drop(columns=['Total
expenditure','GPD','percentage
expenditure','Population'])
df_lifeExp_clean

this our code we name the panda as df_lifexp so what's
the error?
```

Result:

The error in your code is a small typo that happens to the best of us!

1. The Main Error: The "GPD" Typo

In your list of columns to drop, you wrote `'GPD'`, but the actual column name in the dataset is `'GDP'`.

When Pandas doesn't find the exact name (including the order of letters), it throws a `KeyError`.

2. Variable Case Sensitivity

You mentioned in your text that you named the dataframe `df_lifexp`, but in your code, you wrote `df_lifeExp` (with a capital E). Python is case-sensitive, so `df_lifeExp` and `df_lifexp` are seen as two different things.

The Corrected Code:

Here is the fixed version. I also added a `try-except` block logic or just the clean list so it runs perfectly:

```
Python

# 1. Strip the whitespace (Great job including this step!)
df_lifeExp.columns = df_lifeExp.columns.str.strip()

# 2. Fix the typo: Changed 'GPD' to 'GDP'
# Also check that your dataframe name matches exactly (df_lifeExp)
df_lifeExp_clean = df_lifeExp.drop(columns=['Total expenditure', 'GDP', 'percentage
expenditure', 'Population'])

# 3. View the result
df_lifeExp_clean.head()
```

Intellectual Control: Was a simple fix in this case and we just changed fixed the letter into it's correct format and printed the results to check if it printed correctly or not

Prompt:

what does the zero in the file i sent mean and do i have to get it's mean if it's zero or blank?

Result:

In data science, there is a huge difference between a **Zero** and a **Blank (NaN)**. Understanding this is vital because handling them incorrectly can ruin your model's accuracy.

Here is the breakdown for your specific dataset:

1. What does "Zero" mean?

In your dataset, a "Zero" is usually a **valid measurement**, not a mistake.

- `infant deaths` or `Measles` = **0**: This means that in that specific year, the country successfully recorded zero deaths or zero cases of that disease. This is "Good Data"—it shows the country is doing well.
- `Income composition of resources` = **0**: This is more suspicious. Since this is an index from 0 to 1, a 0 would mean the country has absolutely no human development, which is rare.
- `Schooling` = **0**: This means the average person has zero years of education.

Do you need to get the mean for Zeros?

No. Do not replace 0 with the mean for columns like `infant deaths`, `Measles`, or `under-five deaths`. If you change these zeros to the average (which is around 30 for infant deaths), you are "punishing" healthy countries by telling the model they had 30 deaths when they actually had zero.

2. What does "Blank" (NaN) mean?

A "Blank" is **Missing Data**. It means the information was never collected or recorded for that country in that year.

- Example:** Your `Alcohol` column has 194 blanks, and `Hepatitis B` has 553 blanks.

Do you need to get the mean for Blanks?

Yes (or Drop them). You cannot feed "Blanks" into a machine learning model; the code will crash. You have two professional engineering choices:

- Imputation (Mean/Median):** You calculate the average of the column and fill the blanks with that number. This is good if you want to keep as much data as possible.
- Dropping:** You delete the entire row if it has a blank. This is what I suggested earlier because it's the "cleanest" for a small team, provided you still have enough rows left (you have ~2,000, which is plenty).

Comparison Table for your Technical Report

Feature	Value	What it represents	What you should do
<code>infant deaths</code>	0	Actual recorded success.	Leave it as 0.
<code>Alcohol</code>	Blank	Missing information.	Fill with mean OR Drop row.
<code>Life expectancy</code>	Blank	Missing Target variable.	Always Drop the row (Never guess the answer).

Export to Sheets



Intellectual Control: Prompt here was only used to understand a concept while importing our data set.

Prompt:

```
avg = df_lifeExp_clean['Life expectancy'].mean()
df_lifeExp_clean =
df_lifeExp_clean.replace(to_replace=0, value=avg)
print(df_lifeExp_clean.isnull().sum())

we want to replace blank by avg... but not working an
idea what's error?
```

Result:

I see exactly what is happening! Your logic is on the right track for replacing values, but there are **two critical errors** in how Pandas and Data Science work that are causing your code to fail (and ruin your data).

1. The Pandas Error: Zeros vs. Blanks

In your code, you wrote `to_replace=0`. This tells Pandas to find the **number zero** and replace it.

- A "blank" in Pandas is not a zero; it is a `NaN` (Not a Number).
- By replacing `0`, you accidentally took all the healthy countries that had `0` infant deaths and changed their infant deaths to `69`!
- To replace blanks, you must use the Pandas function `.fillna()`, not `.replace()`.

2. The Data Science Error: The "Global" Average

You calculated the average of `Life expectancy` (which is about 69 years) and applied it to the **entire dataset**.

- This means if the `Alcohol` column had a blank, it is now filled with `69` liters of alcohol.
- If the `Polio` column had a blank, it is now `69%`.
- Every column needs to be filled with **its own average**, not the Life Expectancy average.

The Correct Code (Copy and Paste this)

Here is the professional way to clean this data. We will drop the blanks from the Target (`Life expectancy`), and then fill the blanks in the other columns with their *own* averages.

Python

```
import pandas as pd

# 1. Load and clean column names
df_lifeExp = pd.read_csv('Life Expectancy Data.csv')
df_lifeExp.columns = df_lifeExp.columns.str.strip()

# 2. Drop the columns you don't want
cols_to_drop = ['Total expenditure', 'GDP', 'percentage expenditure', 'Population']
df_lifeExp_clean = df_lifeExp.drop(columns=cols_to_drop)

# -----
```

```
# -----
# THE FIX STARTS HERE
# -----

# 3. DO NOT GUESS THE TARGET: Drop rows where 'Life expectancy' is blank
df_lifeExp_clean = df_lifeExp_clean.dropna(subset=['Life expectancy'])

# 4. FILL THE REST WITH AVERAGES:
# This neat Pandas trick fills the blanks in every remaining column with that spec
df_lifeExp_clean = df_lifeExp_clean.fillna(df_lifeExp_clean.mean(numeric_only=True))

# 5. VERIFY: You should now see all zeros!
print(df_lifeExp_clean.isnull().sum())
```

Intellectual Control: Again we here used AI to understand our mistakes and fix our code as it is a common trend for us

Prompt:

income composition and schooling got blanks we can fill it fine but the thing is they got 0 to which is illogical so how would we fill those ?

Results:

You are 100% correct, and noticing this shows excellent engineering intuition!

You have discovered what we call **"Implicit Missing Data."** Sometimes, whoever collected the data didn't leave the cell blank; they just typed a 0 because they didn't know the answer. As you said, a whole country having an average of exactly 0.0 years of schooling or a 0.0 Human Development Index is illogical.

The Danger of the "Fake Zeros"

If you calculate the average of the `Schooling` column right now, all those fake 0s will drag the average down. Instead of the real average being around 12 years, the 0s might drag it down to 10 years, which ruins your data.

The Professional Fix (Two Steps)

To fix this, we do a very common Data Science trick:

1. Turn the fake 0s into Blanks (NaN).
2. Fill all the Blanks (both the original ones and the new ones) with the real average.

Python

```
import pandas as pd
import numpy as np

# 1. Load data and clean names
df_lifeExp = pd.read_csv('Life Expectancy Data.csv')
df_lifeExp.columns = df_lifeExp.columns.str.strip()
df_lifeExp_clean = df_lifeExp.drop(columns=['Total expenditure', 'GDP', 'percentage'])

# 2. Drop the target blanks
df_lifeExp_clean = df_lifeExp_clean.dropna(subset=['Life expectancy'])

# -----
# HOW TO FIX THE ILLOGICAL ZEROS
# -----

cols_with_fake_zeros = ['Schooling', 'Income composition of resources']
```

```
for col in cols_with_fake_zeros:
    # Step A: Replace the number 0 with NaN (a blank)
    # We use np.nan from the numpy library to represent a true blank
    df_lifeExp_clean[col] = df_lifeExp_clean[col].replace(0, np.nan)

    # Step B: Calculate the TRUE average (Pandas automatically ignores NaNs!)
    true_avg = df_lifeExp_clean[col].mean()

    # Step C: Fill all the blanks with this true average
    df_lifeExp_clean[col] = df_lifeExp_clean[col].fillna(true_avg)

print("Illogical zeros fixed successfully!")
```

Intellectual Control: Here we wanted to understand how would we remove the illogical zeros from this prompt we decided on using the method of turning all zeros to blanks in those 2 features not using the given code but a line as was done in our code.

Prompt:

```
y=df_lifeExp_clean['Life expectancy']
features = [
    'Adult Mortality',
    'infant deaths',
    'Alcohol',
    'Hepatitis B',
    'Measles',
    'BMI',
    'under-five deaths',
    'Polio',
    'Diphtheria',
    'HIV/AIDS',
    'thinness 1-19 years',
    'thinness 5-9 years',
    'Income composition of resources',
    'Schooling',
    'Year'
]
```

for feature in features:

```
F_train, F_test, y_train, y_test = train_test_split(feature,
y, test_size=0.3, random_state=42, stratify=y)
Lreg=LinearRegression(fit_intercept=True)
Lreg.fit(F_train,y_train)
Lreg.coef_
Lreg.intercept_
print("--- Regression Results ---")
print(f"R-squared Score: {r2_score(y_test, y_train):.4f}")
print(f"Mean Squared Error (MSE):
{mean_squared_error(y_test, y_train):.4f}")
```

what's the error here it says error in the ftrain line stuff

Result:

The error on the `train_test_split` line is happening because you are passing the **name** of the column (a piece of text) instead of the **actual data** inside that column.

Here are the 4 specific things causing your code to crash:

1. Data vs. Name

In your loop, `feature` is just a string (like `"Adult Mortality"`). `train_test_split` needs the actual numbers.

- **Wrong:** `train_test_split(feature, ...)`
- **Right:** `train_test_split(df_lifeExp_clean[[feature]], ...)` (The double brackets `[[ ]]` are important because Scikit-Learn expects a 2D table for X).

2. The `stratify` Error

The `stratify` parameter is used in **Classification** to make sure you have an equal mix of "Dogs vs Cats" in both sets. Since `Life expectancy` is a continuous number (65.2, 70.1, etc.), you cannot stratify it.

- **Fix:** Remove `stratify=y`.

3. Missing Predictions

To calculate the R^2 score and MSE, you must compare the **Actual Answers** (`y_test`) with the **Model's Guesses** (`y_pred`). You haven't told the model to guess anything yet!

- **Fix:** Add `y_pred = Lreg.predict(F_test)`.

4. Scoring logic

In your print statement, you wrote `r2_score(y_test, y_train)`. This will crash because

`y_test` has 30% of the rows and `y_train` has 70%. You can't compare two lists of different sizes.

Python

```
# Your target
y = df_lifeExp_clean['Life expectancy']

# Your feature list (Note: 'thinness 1-19 years' usually has 2 spaces in this d
features = [
    'Adult Mortality', 'infant deaths', 'Alcohol', 'Hepatitis B',
    'Measles', 'BMI', 'under-five deaths', 'Polio', 'Diphtheria',
    'HIV/AIDS', 'thinness 1-19 years', 'thinness 5-9 years',
    'Income composition of resources', 'Schooling', 'Year'
]

for feature in features:
    # 1. Pass the actual data frame column, not just the name
    # 2. Removed stratify because this is Regression
    F_train, F_test, y_train, y_test = train_test_split(
        df_lifeExp_clean[[feature]], y, test_size=0.3, random_state=42
    )
```

```
# 3. Train the model
Lreg = LinearRegression(fit_intercept=True)
Lreg.fit(F_train, y_train)

# 4. Generate the "guesses" (predictions)
y_pred = Lreg.predict(F_test)

# 5. Evaluate comparing Actual vs. Predicted
print(f"--- Results for {feature} ---")
print(f"R-squared Score: {r2_score(y_test, y_pred):.4f}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print("-" * 30)
```

Intellectual Control: We fixed our errors and understood them by this prompt and normalized our data in our main code thus finishing this part

Prompt:

```
y=df_lifeExp_clean['Life expectancy']
features = [
    'Adult Mortality',
    'infant deaths',
    'Alcohol',
    'Hepatitis B',
    'Measles',
    'BMI',
    'under-five deaths',
    'Polio',
    'Diphtheria',
    'HIV/AIDS',
    'thinness 1-19 years',
    'thinness 5-9 years',
    'Income composition of resources',
    'Schooling'
]
print("--- Linear Regression Results ---")
```

```
i=0
j=0
for feature in features:
    F_train, F_test, y_train, y_test =
    train_test_split(df_lifeExp_clean[[feature]], y,
    test_size=0.3, random_state=42)
    Lreg=LinearRegression(fit_intercept=True)
    scaler = StandardScaler()
    F_train_scaled = scaler.fit_transform(F_train)
    F_test_scaled = scaler.transform(F_test)
    Lreg.fit(F_train_scaled, y_train)
    y_pred = Lreg.predict(F_test_scaled)

    fig, axs = plt.subplots(nrows=4, ncols=4, figsize=(70,
    50))
    axs[i,j].scatter(F_train,y_train)
    axs[i,j].plot(F_train,y_pred)
    i=i+1
    j=j+1
```

```
#print("-----")
-----")
```

```
#print(f"Testing Feature: {feature}")
#print(f"R-squared Score: {r2_score(y_test,
y_pred):.4f}")
#print(f"Mean Squared Error (MSE):
{mean_squared_error(y_test, y_pred):.4f}")
```

we want to draw each in a seprate figure if you know
what i mean before we see the last thing

Results:

I definitely know what you mean! You want a **visual gallery** where you can see the "line of best fit" for every single feature side-by-side to see which ones actually look like a straight line and which ones look like a "cloud" of random dots.

There are three technical "bugs" in your current plotting logic that will prevent this from working:

1. **The "Poster" Problem:** You put `plt.subplots(4, 4)` *inside* the loop. This tells Python to create 14 giant grids of 16 empty plots every time it sees a new feature. You'll end up with hundreds of empty boxes!
2. **The Diagonal Problem:** By doing `i=i+1` and `j=j+1` at the same time, your code only tries to draw on the diagonal (0,0), (1,1), (2,2). It will miss the rest of the grid.
3. **The Size Mismatch:** You are trying to plot `y_pred` (which is 30% of your data) against `F_train` (which is 70%). Python will throw a "Shape Mismatch" error.

```
# --- The Plotting Fix ---
# Plot the real data points (we use the test set for a cleaner look)
axs[row, col].scatter(F_test, y_test, color='blue', alpha=0.3, s=10)

# Plot the Regression Line
# We plot F_test against y_pred (both are the same 'test' size)
axs[row, col].plot(F_test, y_pred, color='red', linewidth=2)

axs[row, col].set_title(f"Life Exp vs {feature[:15]}...") # Shorten long name
axs[row, col].set_xlabel(feature)
axs[row, col].set_ylabel("Life Expectancy")

# 3. Hide the 2 empty boxes at the end (since 14 features < 16 slots)
axs[3, 2].axis('off')
axs[3, 3].axis('off')

plt.show()
```

```
# 1. Initialize the grid ONCE (Before the loop)
# 4x4 gives us 16 slots, which is perfect for your 14 features
fig, axs = plt.subplots(nrows=4, ncols=4, figsize=(20, 20))
fig.subplots_adjust(hspace=0.4, wspace=0.3) # Give the plots some breathing room

# 2. Use 'enumerate' to get both the index (0, 1, 2...) and the feature name
for idx, feature in enumerate(features):
    # Calculate row and column based on the index
    row = idx // 4
    col = idx % 4

    # --- Your existing Regression Logic ---
    F_train, F_test, y_train, y_test = train_test_split(df_lifeExp_clean[[feature]],
                                                        y=df_lifeExp_clean['Life_Expectancy'],
                                                        test_size=0.3, random_state=42)
    scaler = StandardScaler()
    F_train_scaled = scaler.fit_transform(F_train)
    F_test_scaled = scaler.transform(F_test)

    Lreg = LinearRegression()
    Lreg.fit(F_train_scaled, y_train)
    y_pred = Lreg.predict(F_test_scaled)
```

Why this works:

- **The Index Trick (// and %):** This is a classic coding pattern. `idx // 4` (integer division) tells us which row we are on, and `idx % 4` (remainder) tells us which column. It fills the grid perfectly from left to right.
- **Scatter vs. Line:** We scatter the original `F_test` (unscaled) against the original `y_test` so the axes show real years/units. Then we plot the `y_pred` line on top.
- **Alpha & Size:** I added `alpha=0.3` and `s=10`. This makes the dots slightly transparent so you can see where the data is "thickest."

Intellectual Control: We used this prompt because we had an error were all our graphs would be printed on a single graph thus we used ai to help us here and understood how and why it did it that way.

Prompt:

can you explain the x-axis like in adult mortality if iut's 700
what does that mean

Result:

In the context of the **WHO Life Expectancy dataset**, the values you see on the x-axis for

Adult Mortality are not percentages or raw counts of people.

The Definition

Adult Mortality is defined as the **probability of dying between the ages of 15 and 60 per 1,000 population**.

So, if you look at your scatter plot and see a data point at **700** on the x-axis:

- It means that in that specific country/year, for every 1,000 people who reached age 15, **700 of them died** before reaching age 60.
- Mathematically, that is a **70% chance** of dying during adulthood.

Understanding the Scale

To put that into perspective for your project analysis:

Value on X-axis	What it implies
Below 100	Very low mortality (Common in developed nations like Japan or Switzerland).
150 - 300	Average to high mortality (Developing nations).
Above 500	Extreme mortality crisis (Usually seen in countries during major conflicts or the peak of the HIV/AIDS epidemic).

Intellectual Control: This was a simple prompt used to understand the data we got on our graph as we got lost and wanted to understand what it meant.

Prompt:

```
degree=2

print("--- Polynomial Regression Results ---")

fig, axs = plt.subplots(nrows=4, ncols=4, figsize=(25,
20))
fig.subplots_adjust(hspace=0.4, wspace=0.3)

for index, feature in enumerate(features):
    row = index // 4
    col = index % 4

    F_train, F_test, y_train, y_test =
train_test_split(df_lifeExp_clean[[feature]], y,
test_size=0.3, random_state=42)
    preg = LinearRegression(fit_intercept=True)
    scaler = StandardScaler()
```

```
    axs[row, col].scatter(F_test, y_test, color='blue',
alpha=0.3, s=10)
    axs[row, col].plot(F_test, y_pred, color='red',
linewidth=2)
    axs[row, col].set_title(f"Life Exp vs {feature[:15]}...")
    axs[row, col].set_xlabel(feature)
    axs[row, col].set_ylabel("Life Expectancy")

    print("-----")
    print(f"Testing Feature: {feature}")
    print(f"R-squared Score: {r2_score(y_test, y_pred):.4f}")
    print(f"Mean Squared Error (MSE):
{mean_squared_error(y_test, y_pred):.4f}")
```

```
poly = PolynomialFeatures(degree=degree)
F_train_poly = poly.fit_transform(F_train)
F_test_poly = poly.transform(F_test)

F_train_scaled = scaler.fit_transform(F_train_poly)
F_test_scaled = scaler.transform(F_test_poly)
preg.fit(F_train_scaled, y_train)
y_pred = preg.predict(F_test_scaled)

x_range = np.linspace(F_test.min(), F_test.max(),
100).reshape(-1, 1)
x_range_poly = poly.transform(x_range)
x_range_scaled = scaler.transform(x_range_poly)
y_range_pred = preg.predict(x_range_scaled)
```

```
axs[3, 2].axis('off')
axs[3, 3].axis('off')

plt.show()
```

weird straight lines instead of the polynomial expected
line and it says in printing features are not found

Results:

The "weird straight lines" and the "feature not found" errors are coming from two specific spots in your code. Since you are now in **Polynomial land**, the way you handle the plotting data and the feature names has to be slightly more precise.

1. Why you see "weird lines" (The Plotting Bug)

Look at this line in your code:

```
axs[row, col].plot(F_test, y_pred, color='red', linewidth=2)
```

Even though you calculated a beautiful smooth curve called `y_range_pred`, you aren't using it! You are plotting the **raw, unsorted** test data. Because `F_test` isn't in order (it's random samples), the computer tries to connect the dots in the order they appear in the list, creating a "shattered glass" or "zig-zag" look that looks like straight lines everywhere.

The Fix: Use the `x_range` and `y_range_pred` you already calculated.

2. The "Feature Not Found" Error

This usually happens because of the **thinness** column. In the original WHO dataset, there are **two spaces** between "thinness" and "1-19 years". If your list only has one space, Python will crash.

```
degree = 2

print("--- Polynomial Regression Results ---")

fig, axs = plt.subplots(nrows=4, ncols=4, figsize=(25, 20))
fig.subplots_adjust(hspace=0.4, wspace=0.3)

for index, feature in enumerate(features):
    row = index // 4
    col = index % 4

    # Fix for potential column name spacing issues
    try:
        X_data = df_lifeExp_clean[[feature]]
    except KeyError:
        # Fallback if there's a typo in the feature list vs dataframe columns
        print(f"Skipping {feature}: Column name not found in DataFrame.")
        continue

    F_train, F_test, y_train, y_test = train_test_split(X_data, y, test_size=0.3
```

```
poly = PolynomialFeatures(degree=degree)
scaler = StandardScaler()
preg = LinearRegression(fit_intercept=True)

# Transform and Scale
F_train_poly = poly.fit_transform(F_train)
F_test_poly = poly.transform(F_test)
F_train_scaled = scaler.fit_transform(F_train_poly)
F_test_scaled = scaler.transform(F_test_poly)

# Train and Predict
preg.fit(F_train_scaled, y_train)
y_pred = preg.predict(F_test_scaled)

# --- THE PLOTTING FIX ---
# 1. Create a smooth X range for the curve
x_range = np.linspace(F_test.min(), F_test.max(), 100).reshape(-1, 1)
# 2. Transform the range so the model can read it
x_range_poly = poly.transform(x_range)
x_range_scaled = scaler.transform(x_range_poly)
# 3. Predict the Y values for that smooth curve
```

```
    y_range_pred = preg.predict(x_range_scaled)

    # Plot raw data dots
    axs[row, col].scatter(F_test, y_test, color='blue', alpha=0.3, s=10)

    # THE KEY CHANGE: Plot the smooth range, not the raw test data!
    axs[row, col].plot(x_range, y_range_pred, color='red', linewidth=3)

    axs[row, col].set_title(f"Poly Fit: {feature[:15]}")
    axs[row, col].set_xlabel(feature)
    axs[row, col].set_ylabel("Life Expectancy")

    print("-----")
    print(f"Testing Feature: {feature}")
    print(f"R-squared Score: {r2_score(y_test, y_pred):.4f}")
    print(f"MSE: {mean_squared_error(y_test, y_pred):.4f}")

# Hide unused plots
axs[3, 2].axis('off')
axs[3, 3].axis('off')
```

Why this fixes the visual:

In your old code, `y_pred` was mapped to `F_test`. Since `F_test` might go `[10, 50, 20, 80]`, the line would jump back and forth.

By using `x_range` (which goes `[10, 11, 12, 13...]`), the red line is forced to draw in a smooth, continuous path from left to right.

Intellectual Control: After we built our Polynomial Regression we had an error where it showed a linear regression. We used this prompt to fix it and understand as our code was adjusted from the linear regression model we made thus had many errors at the start.

Prompt:

```
y_classifictaion = df_lifeExp_clean.columns['status'].map(lambda x:1 if x ==  
'Developing' else x = 0)
```

i wanna map Developing to 1 and Developed to 0

Results:

Here are the issues in your original line:

1. `df.columns['status']` is incorrect. To access a column, use `df['status']`.
2. `else x = 0` is invalid syntax inside a `lambda`. You just need to return the value: `else 0`.

✓ Corrected Version of Your Code:

python



```
1 y_classification = df_lifeExp_clean['status'].map(lambda x: 1 if x == 'Developing' else 0)
```

💡 Better & More Idiomatic Alternatives:

1. Using a dictionary with `.map()` (Recommended for readability & speed):

python



```
1 y_classification = df_lifeExp_clean['status'].map({'Developing': 1, 'Developed': 0})
```

2. Using boolean comparison (Most efficient for binary targets):

python



```
1 y_classification = (df_lifeExp_clean['status'] == 'Developing').astype(int)
```

(This checks if the value equals 'Developing', returns `True/False`, and converts to `1/0`)

All three will give you the exact result you want. The boolean approach or dictionary map are generally preferred in pandas for performance and cleaner code.

Intellectual property: We did our mapping ourselves and the code outputted incorrectly thus we used this prompt to understand what to fix